Phát hiện và kiểm chứng lỗ hổng bảo mật ở cấp repository bằng hệ thống đa tác tử kết hợp phân tích tĩnh và sinh Proof-of-Vulnerability

Repository-Level Vulnerability Detection and Validation Using a Multi-Agent System with Static Analysis and Proof-of-Vulnerability Generation

Phạm vi: nghiên cứu hệ thống đa tác tử có khả năng khám phá repository, truy vết source-to-sink, sinh giả thuyết lỗ hổng và tạo Proof-of-Vulnerability (PoV) được kiểm chứng bằng thực thi.

1. Tên đề tài và mô tả bài toán

Tên tiếng Việt: Phát hiện và kiểm chứng lỗ hổng bảo mật ở cấp repository bằng hệ thống đa tác tử kết hợp phân tích tĩnh và sinh Proof-of-Vulnerability.

Tên tiếng Anh: Repository-Level Vulnerability Detection and Validation Using a Multi-Agent System with Static Analysis and Proof-of-Vulnerability Generation.

Bài toán nghiên cứu là xây dựng một hệ thống tự động phân tích toàn bộ repository thay vì chỉ nhận một hàm đã được định vị. Hệ thống phải xác định các điểm vào có thể bị tác động, truy vết dữ liệu và luồng điều khiển đến thao tác nhạy cảm, nêu nguyên nhân theo CWE, sau đó sinh input hoặc test có thể kích hoạt lỗi. Một phát hiện chỉ được xem là đã kiểm chứng khi PoV tạo ra bằng chứng thực thi phù hợp, chẳng hạn sanitizer report, test failure, taint violation hoặc hành vi khác biệt giữa phiên bản lỗi và phiên bản đã vá.

2. Khoảng trống nghiên cứu

- Các phương pháp LLM hiện thường sinh nhãn hoặc mô tả tự nhiên nhưng chưa tạo được bằng chứng thực thi đáng tin cậy; Revelio cho thấy việc tách hypothesis generation và executable confirmation có thể giảm hallucination, nhưng tập trung chủ yếu vào memory safety trong C/C++.
- Lỗ hổng thực tế thường trải dài qua nhiều file, hàm và module. VulnGym chỉ ra rằng đánh giá đúng nhãn chưa đủ; cần kiểm tra vị trí, critical operation và vulnerability trace.
- Các nghiên cứu multi-agent và static analysis thường được đánh giá riêng, chưa có pipeline thống nhất từ candidate generation, trace construction đến PoV validation.
- Các benchmark hiện chưa đồng thời phản ánh precision, trace completeness, khả năng tái hiện và chi phí xử lý repository.
- Kết quả có thể bị thổi phồng bởi data leakage giữa các phiên bản của cùng repository; cần đánh giá theo repository-disjoint và time-based split.

3. Câu hỏi và giả thuyết nghiên cứu

- RQ1. Hệ thống đa tác tử kết hợp phân tích tĩnh có cải thiện phát hiện lỗ hổng cấp repository so với LLM đơn lẻ và ReAct agent không?
- RQ2. Call graph, data-flow graph và kết quả static analysis ảnh hưởng thế nào đến định vị mã nguồn và độ đầy đủ của vulnerability trace?
- RQ3. Việc phân chia vai trò giữa các agent có cải thiện hiệu quả so với một agent duy nhất không?
- RQ4. PoV generation và execution verification có làm giảm false positive và tăng verified precision không?
- RQ5. Kiến trúc phân tầng mô hình có đạt trade-off tốt hơn giữa recall, thời gian và chi phí không?
- RQ6. Hệ thống có tổng quát hóa sang repository, commit và CWE chưa xuất hiện trong quá trình phát triển không?

Các giả thuyết kiểm định: H1: MAS+SA đạt recall/F1 cao hơn LLM đơn và ReAct; H2: đồ thị chương trình cải thiện localization và trace F1; H3: multi-agent tạo trace đầy đủ hơn single-agent; H4: PoV validation làm giảm false-positive rate; H5: execution feedback tăng PoV success rate và giảm time-to-first-valid-PoV; H6: kiến trúc phân tầng giảm cost per confirmed vulnerability trong khi recall không giảm đáng kể; H7: hiệu năng giảm có kiểm soát trên repository hoặc CWE chưa thấy.

4. Contribution dự kiến

- Đề xuất kiến trúc multi-agent gồm Planner/Coordinator, Repository Retriever, Static-Analysis Interpreter, Vulnerability Analyst, Trace Builder, PoV Generator và Deterministic Verifier.
- Thiết kế schema bằng chứng thống nhất gồm CWE, file/dòng, entry point, critical operation, source-to-sink trace, precondition, PoV và kết quả thực thi.
- Xây dựng cơ chế phối hợp và xếp hạng candidate dựa trên reachability, static-analysis evidence, severity và khả năng kiểm chứng.
- Thực hiện đánh giá có kiểm soát về detection, localization, trace, PoV, false positive, thời gian và chi phí; kèm ablation study và kiểm tra data leakage.

5. Dataset và chiến lược chia dữ liệu

Dataset	Vai trò	Thông tin chính	Cách dùng

VulnGym	Dataset chính	408 entries, 23 repository; entry point, critical operation, trace, human verification	Đánh giá khám phá, định vị, trace và end-to-end; ưu tiên verify=1
CyberGym	PoV validation	Bài toán tái hiện lỗ hổng trên codebase chưa vá bằng PoC/test	Đánh giá PoV success, reproducibility và execution feedback
JITVUL	Tổng quát hóa commit	879 CVE, 1.758 cặp vulnerable–fixed commit, 91 CWE	Đánh giá JIT detection và phân biệt trước/sau vá
CWE-Bench-Java hoặc ReposVul	Dataset phụ	Java CVE có build/patch hoặc repository-level C/C++	Kiểm tra chéo theo ngôn ngữ và giảm phụ thuộc một benchmark

Chia dữ liệu: (a) repository-disjoint split để tránh rò rỉ giữa các repository; (b) time-based split để mô phỏng lỗ hổng phát sinh sau thời điểm phát triển; (c) CWE-held-out split để đo khả năng chuyển giao. Không chia ngẫu nhiên theo function nếu các phiên bản của cùng repository xuất hiện ở cả train và test.

6. Kịch bản thực nghiệm và baseline

- S1 End-to-end discovery: agent nhận repository chưa được định vị, tự tìm vulnerability, trace và PoV. Đánh giá trên VulnGym.
- S2 Oracle localization: cung cấp entry point hoặc critical operation, đo khả năng truy vết và sinh bằng chứng. Dùng để tách lỗi "không tìm thấy vùng mã" khỏi lỗi "không hiểu lỗ hổng".
- S3 PoV generation: cung cấp vulnerability description và codebase chưa vá, yêu cầu sinh test/input; đánh giá trên CyberGym.
- S4 JIT detection: chỉ cung cấp diff/commit và context liên quan, phân biệt vulnerable và fixed version trên JITVUL.
- S5 Ablation: lần lượt bỏ static analysis, call graph, data-flow, coordinator, ranking, self-reflection và execution feedback.
- S6 Cost-aware orchestration: so sánh dùng model mạnh cho toàn pipeline với mô hình rẻ cho scan và mô hình mạnh cho PoV/verification.

Baseline	Mô tả	Mục đích
B0 SAST	CodeQL hoặc Semgrep không có LLM	Đường cơ sở công cụ truyền thống
B1 LLM	LLM đọc context được chọn, không dùng tool	Đo năng lực suy luận thuần
B2 ReAct	Một agent có search, shell và iterative reasoning	So sánh với agent đơn
B3 Agent+SA	Một agent nhận kết quả static analysis	Đo lợi ích của SA
B4 Multi-agent	Nhiều agent nhưng không có PoV verifier	Đo lợi ích của phối hợp
B5 Đề xuất	Multi-agent + SA + trace + PoV + verifier	Mô hình đầy đủ
B6 Strong-model-only	Model mạnh chạy xuyên suốt repository	So sánh chi phí và hiệu năng

7. Độ đo đánh giá

- Detection: vulnerability-level precision, recall, F1, false-positive rate, false-negative rate và Recall@K. Một TP cần đúng vulnerability và có bằng chứng/trace phù hợp.
- Localization: exact file/function/line match, line ±3/±5, Top-k localization và khoảng cách dòng tới ground truth.
- Trace: node/edge coverage, trace precision, recall, F1, path-order accuracy và trace completeness.
- PoV: PoV success rate, reproduction rate, verified true-positive rate, verified false-positive rate, time-to-first-valid-PoV, số lần thử và tính tái lập qua 3–5 lần chạy.
- Hiệu năng: thời gian/repository, token, số lượt gọi model, chi phí, cost per confirmed vulnerability và confirmed vulnerabilities per hour/dollar.
- Thống kê: báo cáo trung bình, median, độ lệch chuẩn hoặc bootstrap 95% CI; dùng McNemar cho kết quả phát hiện cặp đôi và Wilcoxon cho thời gian/chi phí theo repository.

8. Tài liệu tham khảo chính

[1] Hou, Y. et al. (2026). Revelio: Cost-Efficient Agentic Memory Safety Vulnerability Detection for Repository-Scale Codebases. arXiv:2606.22263. https://arxiv.org/abs/2606.22263

[2] Ji, K. et al. (2026). VulnGym: Benchmarking Coding Agents for Repository-Level Vulnerability Detection. arXiv:2608.02001. https://arxiv.org/abs/2608.02001

[3] Yildiz, A. et al. (2025). Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories. ACL 2025, 30848–30865. https://aclanthology.org/2025.acl-long.1490/

[4] Wen, X.-C. et al. (2024). Collaboration to Repository-Level Vulnerability Detection. ISSTA/ECOOP 2024 Doctoral Symposium. DOI: 10.1145/3650212.3685562.

[5] Wang, Z. et al. (2024). ReposVul: A Repository-Level High-Quality Vulnerability Dataset. arXiv:2401.13169. https://arxiv.org/abs/2401.13169

[6] Li, Z., Dutta, S., & Naik, M. (2024). LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. CWE-Bench-Java. https://github.com/iris-sast/cwe-bench-java

[7] CyberGym Observatory (2026). Vulnerability Reproduction and End-to-End Benchmarks. https://www.cybergym.io/